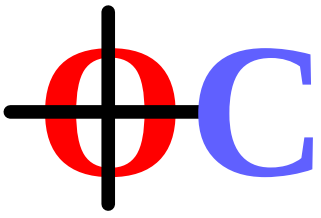
The

Coda-C

Library

Object-Oriented
Programming in C



Stephen M. Jones

This document describes: Coda-C Library Version 3.0. 09/01/2026

www.coda-c.com

• Just like all software, there are NO WARRANTIES.

This document was typeset with GNU troff (groff) running on MAC OS X.

Credits

I'm deeply thankful to Lynn W. Keys, my lifelong partner, who's assistance made this writing and in fact the development of the underlying software possible. Her unique perspective, technical writing skills, mastery of the English language, and uncanny ability to quickly reveal the undocumented features of my bullet proof software, have proved invaluable to my development of software and the related documentation.

I would like to thank Brian W. Kernighan and Dennis M. Ritchie, for providing an elegant and powerful language (C) that has inspired me and provided the means to act on that inspiration.

Create **O**bjects, **D**ictionaries & **A**rrays in **C**.

Payload: C header *coda-c.h*, dynamic library *coda-c*, and header generator *coda-h*.

Objects

Coda-C Objects are allocated with metadata supporting retain counting, class, and size. In 'C' object classes are pointer type variables. To refer to a character object you would use 'Char name;' and not 'Char *name;'. To refer to a general object you would use 'Obj name;' where 'Obj' is a void pointer.

The most primitive object is classless and is created with *alocO(size)*, and like all objects, starts with a retain count of 1. An object's retain count may be incremented with *keepO(obj)*. An object's retain count may be decreased with *freeO(obj)* and, if zero, the object is destroyed. All objects have basic properties that can be retreived with functions, like *sizeO(obj)* to determine the segment size and *countO(obj)* to obtain the retain count.

Classes

All defined classes either inherit from another class or a root pseudo class. Creating subclasses may add additional data or be transparent adding only new virtual functions. For example: ConstChar is a transparent subclass of Char used to identify special string objects created with Os("String") that are created from C string constants. Objects of fixed size are created with *newO(class)*, such as the dynamic Array or Dictionary. Objects with a variable size are created with *newOC(class,count)*, (Char or Pointer), e.g., a Pointer object with 7 pointers could be created with *newOC(Pointer,7)*. Classed objects have additional properties, such as *isa_(obj,class)* to determine if an object inherits from a class and *kindO(obj)* to get the class string name.

Signatures

For the purposes of ***Coda-C***, a signature is a typedef of how to call a virtual function along with its return value. Each signature requires a unique signature key string constant. The 'sig_()' macro generates a global key.

```
typedef Obj OSig(subKey)(Obj dict,char *key); sig_(subKey);
typedef Obj OSig$(subKey)(char *key); // is short version of the above
```

*Where **Coda-C** defines OSig(expr) as (*expr) which will create a C call type, and sig_(expr) will create the signature key. Signatures are required to call object related functions that are not known at compile time.*

Virtual calls *obj_(...) super_(...)*

With signatures defined you would actually call an object function as follows:

```
Obj myobject= obj_(subKey,dict1,"The_Key");
```

The 'super_' call has the same format but is only usable inside a class definition.

Note: The example above demonstrates a virtual call, but typically dictionary functions could be called directly in one of the following formats:

```
Obj myobject= Dictionary_subKey(dict1,"The_Key");
Obj myobject= Dict_sub(dict1,"The_Key"); // shorthand
```

*The macro 'void * OResponds(Obj obj, token SIG);' is used to determine if an object supports the given signature 'sig_'+SIG. The macro 'OHasSuper()' does the same for super class functions.*

Core Classes:

Class: *Char*, *ConstChar*

Probably the most useful object class is Char or the string class. The most common methods for creating object strings are:

```
newOC(Char,64); // create a 64 byte string object.
Char_Value(char *string) ; // create object string from C string.
Char_F(char *cs,...) ; // create object string from printf() style input.
Os("String!"); // Or an immutable string constant. Class ConstChar.
```

Note: strings objects are also C char types and may used for any normal C string functions.*

Class: *Pointer*

The Pointer object is a simple container of pointers(void), or like C void *myPointers[20];. Unlike the plain-C version, the number of pointers can be retreived with Pointer_count();. This object is best at passing around distilled results from one place to another.*

Class: *Dictionary*

The dynamic dictionary is a sort of like an array, except you access items with a word or string rather than an index. This concept is fundamental to many programming languages because referring to things by name can be easier to keep up with. The dictionary retains ownership of objects and copies keys.

```
newO(Array);
void Dictionary_setKey(Dictionary dict,char *key,Obj obj);
void Dictionary_takeKey(Dictionary dict,char *key,Obj obj);
Obj  Dictionary_subKey(Dictionary dict,char *key);
void Dictionary_removeKey(Dictionary dict,char *key);
int Dictionary_count(Dictionary dict);
Pointer Dictionary_AllKeys(Dictionary dict);
void Dictionary_set_name(Dictionary dict,char* name);
char* Dictionary_name(Dictionary dict);
```

Class: *CDictionary*

This class uses 'Dictionary' functions but works with raw pointers instead of Objects.

Class: *Keyword*

This is a public class designed to work with 'Dictionary' contents.

Class: ***Array***

*The dynamic array performs the basic function like a simple array of pointers, or like C void *list[50], but has one very cool property: it will dynamically expand or contract the array to fit your data. You don't have to worry about capacity. You can add to the beginning, middle, or end. You can only add Objects to an Array because it retains the objects, releasing them when the array is destroyed.*

```
newO(Array);
int Array_count(Array array);
Obj Array_addObject(Array array,Obj obj);
Obj Array_subInt(Array ptr,int ix);
Array Array_NewBlock(Array proto,int count,pointer block);
void Array_insertAt(Array array,Obj obj,int ix);
void Array_removeAt(Array array,int dix);
void Array_removeLast(Array array);
void Array_replaceAt(Array array,int ix,Obj obj);
int Array_removeObject(Array array,Obj obj);
void Array_removeAll(Array array);
void Array_set_name(Array array,char* name);
char* Array_name(Array array);
Char Array_Join(Array array,char *delimiter);
pointer Array_rawAddress(Array ptr);
```

Note: The Array class will accept NULL pointers.

Class: ***CArray***

This class uses 'Array' functions but works with raw pointers instead of objects.

Class Creation

To create a new Coda-C class, you need a class name (ex. MyClass), a structure or basic storage type (ex. struct MyClass_ or short), and the name of the superclass (ex. Root). The C instructions below will create a new class:

```
struct MyClass_ { int code; char *name; }; // example

#define class MyClass
CodaClassZerosC();
CodaClass(MyClass,struct MyClass_,Root);
#undef class // MyClass
```

The code above uses the macro:

```
#define CodaClassZerosC() CodaClassZeros(dtor,itor,kize,etor,ekeep,bits)
```

In this example, this macro expands to:

```
enum {
   MyClass_dtor=0,
   MyClass_itor=0,
   MyClass_kize=0,
   MyClass_etor=0,
   MyClass_ekeep=0,
   MyClass_bits=0, };
```

This defines the basic parameters of a class, where 'dtor', 'itor', 'etor', and 'ekeep' are function pointers, and 'kize' and 'bits' are integers. When function pointers are defined as actual functions: dtor is the object destructor, itor is the object initializer, etor and ekeep are used for containers objects and are called to release and keep contained objects, The 'kize' integer is used to convert newOC() to reserving 'count' additional bytes to an object instead of arraying the object. The 'bits' integer is used to mark special root classes or mark a subclass as transparent.

If a class needs a function pointer like destructor, the destructor C function is defined and 'dtor' will be removed from the CodaClassZeros() macro, as shown below:

```
struct MyClass_ { int code; char *name; }; // example
CodaClassDef(MyClass,struct MyClass_,Root); // for dtor's 'self'

#define class MyClass
CodaClassZeros(itor,kize,etor,ekeep,bits); // dtor removed
void MyClass_dtor(MyClass self) { free(self->name); }
CodaClass(MyClass,struct MyClass_,Root);
#undef class // MyClass
```

To specify integer values, enums or #defines can be used. For example:

```
#define MyClass_kize sizeof(struct MyClass_)

enum { MyClass_bits=bits_Trans }; // Or
CodaClass_Trans(); // defines this for you
```

This allow the 6 class parameters to be setup. Generally class functions, signatures, methods, properties etc. will also be required.

```
class Char_Value(const char *string) { // create a new object string.
   if (!string) return(0);
   char *cp=newOC(Char,cs_length(string)+1);
   cs_strcopy(cp,string); return(cp);
   }

method$(Char,xmlTag) { return Os("string"); } // Expands to:

static Char CLASS_xmlTag(Char self) { return Os("string"); } $boot1(xmlTag);
   // where $boot1 registers a virtual function.

// The following macros generate get/set functions for objects.
getter$(type,var,body)
setter$(type,var,body)
property$(type,var,body1,body2)

property$_(type,var) // simple value      get/set
property$$(type,var) // subclassed value  get/set
propertyO_(type,var) // object value      get/set
propertyO$(type,var) // subclassed object get/set

// Generate functions like:

type CLASS_var(CLASS self) { return self->var; } // getter
void CLASS_set_var(CLASS self,type value) { self->var=value; } // setter

// Note: "$()" and "_" macros
#define $(sig,...) OBind1_2M(class,sig)(class self,##__VA_ARGS__) // shorthand
#define _ self-> // class member shorthand
```